

Documentação de Arquitetura de Machine Learning

Esta documentação detalha a especificação técnica e arquitetural do pipeline batch de analytics de clientes implementado no arquivo `ml_churn_rfm.py`. O sistema visa calcular o score RFM de engajamento de compra e estimar a probabilidade de churn de clientes a partir do repositório de dados do banco de Business Intelligence (BI).

1. Visão Geral do Sistema

O ecossistema analítico é estruturado em torno da classe central `KickHubCustomerAnalytics`, encarregada de encapsular e coordenar todo o fluxo informacional. A execução do pipeline ocorre de ponta a ponta de forma sequencial pelo método corporativo `run()`. O ciclo operacional realiza as seguintes macro-etapas fundamentais:

- Estabelecimento de sessão e conexão com o banco de BI.
- Validação estrutural e criação da tabela destino de resultados `ml_cliente_scores`.
- Extração de métricas comportamentais agregadas de clientes e vendas.
- Saneamento e tratamento de consistência dos dados extraídos.
- Cálculo do score RFM estendido ponderado.
- Treinamento supervisionado do modelo de propensão ao churn.
- Persistência definitiva dos scores e classificações no banco de dados.

2. Arquitetura Do Pipeline De Dados

O pipeline de dados e modelagem do projeto está concentrado na classe `KickHubCustomerAnalytics`, responsável por executar todo o processo de análise dos clientes. A execução principal acontece no método `run()`, que organiza o fluxo completo desde a criação da tabela de destino até a gravação dos resultados finais.

```
def run(self) -> None:
    self.ensure_table_schema()

    df = self.extract_features()
    if not df.empty:
        df = self.data_quality_pipeline(df)
        df = self.calculate_customer_score(df)
        df = self.train_churn_model(df)
        self.save_results(df)
```

Esse método representa o encadeamento lógico da arquitetura: primeiro os dados são extraídos, depois tratados, pontuados, avaliados pelo modelo de churn e, por fim, persistidos no banco.

Camada De Origem: Banco BI PostgreSQL

A origem dos dados é o banco BI em PostgreSQL. A conexão é configurada no construtor da classe, usando variáveis de ambiente como BI_HOST, BI_DB, BI_USER e BI_PASSWORD.

```
self.bi_config = {  
    'host': os.getenv('BI_HOST', 'localhost'),  
    'port': os.getenv('BI_PORT', '5432'),  
    'database': os.getenv('BI_DB', 'kickhub_bi'),  
    'user': os.getenv('BI_USER', 'postgres'),  
    'password': os.getenv('BI_PASSWORD', 'postgres')  
}
```

Essa camada fornece os dados brutos das tabelas bi_clientes e bi_vendas, que são a base para a criação das variáveis analíticas.

Módulo De Extração: Queries SQL E Features RFM

A extração ocorre no método extract_features(). Nele, uma query SQL calcula métricas de comportamento de compra, como frequência, valor total, ticket médio, última compra e primeira compra.

```
WITH metricas_vendas AS (  
    SELECT  
        clienteld,  
        COUNT(id) as frequencia_total,  
        SUM(total) as valor_total,  
        AVG(total) as ticket_medio,  
        MAX(dataVenda) as ultima_compra,  
        MIN(dataVenda) as primeira_compra  
    FROM bi_vendas  
    WHERE status = 'CONCLUIDA'  
    GROUP BY clienteld  
)
```

Essas métricas são transformadas em features RFM:

```
COALESCE(mv.frequencia_total, 0) as frequencia_total,  
COALESCE(mv.valor_total, 0.0) as valor_total,  
COALESCE(mv.ticket_medio, 0.0) as ticket_medio,  
COALESCE(EXTRACT(DAY FROM (NOW() - mv.ultima_compra)), 365) as recencia_dias
```

Aqui, clientes sem compra recebem valores padrão, como 0 para frequência e valor, e 365 dias para recência.

Módulo De Qualidade: Sanitização E Tratamento De Anomalias

O tratamento de qualidade é feito em `data_quality_pipeline()`. Esse módulo remove duplicidades, trata valores nulos, corrige valores negativos e remove outliers.

```
df = df.drop_duplicates(subset=['cliente_id'])  
df = df.fillna(0)
```

Valores negativos são considerados inconsistentes para as métricas do projeto e são ajustados para zero:

```
for col in self.features:  
    inconsistent_mask = df[col] < 0  
    df.loc[inconsistent_mask, col] = 0
```

Também existe tratamento estatístico de outliers usando Z-Score, aplicado sobre `valor_total` e `frequencia_total`:

```
z_scores = np.abs(stats.zscore(df_compradores[['valor_total', 'frequencia_total']]))  
outliers_mask = (z_scores > 3).any(axis=1)
```

Isso evita que clientes com valores extremos distorçam o cálculo do score e o treinamento do modelo.

Módulo De Negócio: Score RFM Ponderado E Normalizado

O score analítico de compra é calculado em `calculate_customer_score()`. O objetivo é transformar as métricas RFM em uma pontuação de 0 a 100.

A frequência, valor monetário e tempo de relacionamento são normalizados com `MinMaxScaler`:

```
scaler = MinMaxScaler(feature_range=(0, 100))
score_f = scaler.fit_transform(df[['frequencia_total']]).flatten()
score_m = scaler.fit_transform(df[['valor_total']]).flatten()
score_l = scaler.fit_transform(df[['tempo_relacionamento_dias']]).flatten()
```

A recência recebe tratamento inverso, pois quanto menor o tempo desde a última compra, melhor o cliente é considerado:

```
score_r = 100 - ((df['recencia_dias'] / max_recencia) * 100)
```

Depois, os scores são combinados com pesos de negócio:

```
W_R, W_F, W_M, W_L = 0.20, 0.35, 0.35, 0.10
df['score_compra_rfm'] = (
    (W_R * score_r) +
    (W_F * score_f) +
    (W_M * score_m) +
    (W_L * score_l)
)
```

Assim, frequência e valor total têm maior impacto no score final.

Módulo De IA: Modelo Preditivo De Churn

O módulo de IA está no método `train_churn_model()`. Ele utiliza um `RandomForestClassifier` para prever a probabilidade de churn.

A variável alvo é criada com base em uma regra de negócio: clientes com mais de 90 dias sem comprar são considerados churn.

```
y = (df['recencia_dias'] > self.churn_threshold_days).astype(int)
```

O modelo usa as features definidas na classe:

```
self.features = [  
    'recencia_dias',  
    'frequencia_total',  
    'valor_total',  
    'ticket_medio',  
    'tempo_relacionamento_dias'  
]
```

Antes do treinamento, os dados são normalizados e divididos em treino e teste:

```
X = df[self.features]  
scaler = MinMaxScaler()  
X_scaled = scaler.fit_transform(X)  
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.2, random_state=42, stratify=y  
)
```

O algoritmo usado é Random Forest:

```
model = RandomForestClassifier(  
    n_estimators=150,  
    max_depth=10,  
    random_state=42,  
    class_weight='balanced'  
)  
model.fit(X_train, y_train)
```

Essa escolha permite lidar com relações não lineares e com possível desbalanceamento entre clientes ativos e clientes em churn.

Módulo De Pós-Processamento: Score Híbrido E Risco

Após o Random Forest gerar a probabilidade inicial de churn, o projeto aplica uma regra híbrida. A probabilidade final combina a previsão do modelo com uma regra baseada na recência.

```
rf_probs = model.predict_proba(X_scaled)[: , 1]
recency_decay = (df['recencia_dias'] / 120).clip(0, 1)
df['probabilidade_churn'] = (
    (rf_probs * 0.3) +
    (recency_decay * 0.7)
)
```

Essa abordagem dá maior peso para a regra de tempo sem compra, mantendo parte da inteligência do modelo. Na prática, o churn final é composto por:

- 30% padrão aprendido pelo modelo
- 70% regra de negócio baseada em recência

Depois disso, a probabilidade é convertida em categorias de risco:

```
def classify_risk(prob: float) -> str:
    if prob >= 0.75: return 'Crítico'
    if prob >= 0.50: return 'Alto'
    if prob >= 0.25: return 'Médio'
    return 'Baixo'
```

Essas categorias facilitam o consumo do resultado por dashboards e análises de negócio.

Camada De Destino: Tabela ml_cliente_scores

A saída do pipeline é salva na tabela ml_cliente_scores, criada no método ensure_table_schema().

```
CREATE TABLE IF NOT EXISTS ml_cliente_scores (
    cliente_id BIGINT PRIMARY KEY,
    recencia_dias INTEGER,
    frequencia_total INTEGER,
    valor_total FLOAT,
    ticket_medio FLOAT,
    tempo_relacionamento_dias INTEGER,
    score_compra_rfm FLOAT,
    probabilidade_churn FLOAT,
```

```
risco_churn VARCHAR(50),
calculado_em TIMESTAMP DEFAULT NOW()
);
```

O método `save_results()` grava os dados finais:

```
columns = [
    'cliente_id', 'recencia_dias', 'frequencia_total', 'valor_total',
    'ticket_medio', 'tempo_relacionamento_dias', 'score_compra_rfm',
    'probabilidade_churn', 'risco_churn'
]
execute_values(cursor, query, records)
```

Essa tabela funciona como camada final de consumo, podendo alimentar dashboards, relatórios e estratégias de retenção de clientes.

3. Fonte de Dados e Engenharia de Features

O ponto de partida dos dados reside em um banco PostgreSQL focado em BI, cujos parâmetros de conectividade são abstraídos em variáveis de ambiente dentro do arquivo `.env`. O fluxo de sessão é provido nativamente pelo método interno `get_bi_connection()`.

As fontes consultadas compreendem prioritariamente as tabelas relacionais:

- `bi_clientes`
- `bi_vendas`

O processo de engenharia de recursos ocorre na rotina `extract_features()`. Os atributos comportamentais de consumo extraídos estão mapeados conceitualmente:

Nome do Atributo	Descrição Técnica / Regra de Cálculo
recencia_dias	Intervalo temporal em dias desde a última transação comercial concluída pelo cliente.
frequencia_total	Contagem consolidada de transações faturadas com sucesso ao longo do ciclo de vida.
valor_total	Volume monetário bruto transacionado e acumulado pelo respectivo cliente.

ticket_medio	Indicador de ticket médio, computado pela razão entre o valor_total e a frequencia_total.
tempo_relacionamento_dias	Métrica de tempo de casa (tenure), calculada a partir da primeira compra ou da criação do registro cadastral.

4. Pipeline de Qualidade dos Dados (Data Quality)

Precedendo a etapa estatística e preditiva, o ecossistema executa um saneamento de integridade por meio da função `data_quality_pipeline()`. Esse componente assegura que desvios ou ruídos operacionais não corrompam a performance dos modelos. As operações englobam:

- **Deduplicação Estrutural:** Eliminação de entradas duplicadas tomando como chave única o identificador corporativo `cliente_id`.
- **Tratamento de Missing Values:** Imputação e preenchimento de campos nulos ou ausentes com o valor numérico padrão 0.
- **Sanidade de Valores:** Detecção e retificação de registros negativos incoerentes em variáveis que exigem magnitude estritamente positiva.
- **Exclusão de Outliers:** Filtragem estatística baseada na técnica de Z-Score, removendo registros cujos atributos apresentem $|Z| > 3$. Esta limpeza aplica-se restritamente ao universo de clientes com compras validadas, evitando distorções em clientes de cauda longa.

5. Modelo de Score RFM Estendido

O indicador de engajamento do cliente é construído na rotina `calculate_customer_score()`, com base em uma formulação RFM estendida (incorporando a dimensão L de relacionamento):

- **R (Recência):** Calculada sob lógica inversamente proporcional: quanto menor o hiato temporal sem compras, maior a nota obtida.
- **F (Frequência):** Diretamente proporcional ao número de transações.
- **M (Valor Monetário):** Atrelado ao montante financeiro alocado na plataforma.
- **L (Tempo de Relacionamento):** Mensura a longevidade do vínculo contratual/cadastral do cliente.

Para mitigar disparidades de grandezas, os eixos recebem reescalonamento para o intervalo

dinâmico de 0 a 100 via algoritmo de transformação MinMaxScaler. A ponderação final segue as metas estratégicas de negócio definidas a seguir:

Variável Analítica	Peso Proporcional na Composição
Recência (R)	20%
Frequência (F)	35%
Valor Monetário (M)	35%
Tempo de Relacionamento (L)	10%

Particularidade de Regra: Clientes desprovidos de histórico de transações são parametrizados automaticamente com score de compra fixado em 0, mitigando vieses ocasionados pela antiguidade de cadastros inativos.

6. Modelagem Preditiva de Churn (Machine Learning)

O treinamento do estimador probabilístico de evasão é executado no método `train_churn_model()`. A variável dependente (alvo/target) baseia-se em uma lógica de inatividade delimitada:

- `churn = 1` caso o valor de `recencia_dias` seja estritamente superior a 90 dias.
- `churn = 0` caso o valor de `recencia_dias` seja inferior ou igual a 90 dias.

6.1 Algoritmo e Hiperparâmetros

A escolha técnica de modelagem recai sobre o classificador ensemble `RandomForestClassifier`, justificado pela sua robustez frente a multicolinearidades e capacidade intrínseca de mapeamento de interações não lineares. A configuração estrutural utiliza:

- `n_estimators = 150`: Total de árvores de decisão independentes criadas na floresta.
- `max_depth = 10`: Teto de profundidade das árvores de modo a mitigar cenários de `overfitting`.
- `random_state = 42`: Seed de aleatoriedade fixa garantindo a reprodutibilidade dos ciclos.
- `class_weight = 'balanced'`: Balanceamento automático de pesos associados às classes para contrabalancear assimetrias na distribuição estatística de churners.

6.2 Protocolo de Validação e Treinamento

Antes do acoplamento ao modelo, os atributos são padronizados através do estimador `MinMaxScaler`. O conjunto amostral é segregado adotando a proporção de **80% para treinamento** e **20% para testes de generalização**. A divisão aplica amostragem estratificada

(stratify=y), preservando a taxa original de conversão em ambas as partições.

6.3 Métricas de Avaliação Logadas

O monitoramento contínuo do classificador via subsistema de log, mensurando os seguintes indicadores:

- **Acurácia (Accuracy):** Índice geral que computa a fração de predições corretas sobre o universo total analisado.
- **Precisão (Precision):** Avalia a acurácia das predições positivas, apontando quantos clientes rotulados como churn realmente evadiram.
- **Recall (Sensibilidade):** Percentual de cobertura do modelo, detalhando a capacidade de capturar a totalidade de churners reais da base.
- **F1 Score:** Média harmônica balanceada entre as métricas de Precision e Recall.
- **ROC AUC:** Área sob a curva de operação do receptor, atestando a capacidade do modelo de discriminar probabilisticamente clientes ativos de inativos.

7. Estratégia Híbrida de Churn e Classificação de Risco

Um diferencial marcante da engenharia de decisão é que o índice final de evasão não provém puramente do algoritmo supervisionado. O código implementa uma abordagem híbrida, combinando a componente estatística com uma regra de decaimento puramente linear atrelada à recência cronológica:

$\text{probabilidade_churn} = (\text{rf_probs} * 0.3) + (\text{recency_decay} * 0.7)$

Essa composição estabiliza o score contra desvios extremos do estimador e garante alta interpretabilidade pelas frentes de negócio. A partir do percentual gerado, o método `classify_risk()` categoriza o cliente em quadrantes operacionais:

Intervalo Probabilístico	Classificação do Risco de Churn
0.00 a 0.24	Baixo
0.25 a 0.49	Médio
0.50 a 0.74	Alto
0.75 a 1.00	Crítico

8. Persistência de Resultados e Ciclo de Vida

Os indicadores finais calculados são persistidos na tabela destino `ml_cliente_scores` pela rotina `save_results()`. O esquema armazena o seguinte payload estruturado:

- **cliente_id:** Chave estrangeira de identificação.
- **Métricas descritivas:** `recencia_dias`, `frequencia_total`, `valor_total`, `ticket_medio` e `tempo_relacionamento_dias`.
- **Outputs analíticos:** `score_compra_rfm`, `probabilidade_churn` e `risco_churn`.
- **Metadado temporal:** `calculado_em` (registro da data/hora do processamento).